



M.SC. DATA ANALYTICS & TECHNOLOGIES

DAT7303- Data Mining and Machine Learning

Student Name: Victor Ene

Student Number: 2318217

Module Tutor: Dr Pradeep Hewage

Submitted: 17/04/2025

Word Count: 3555

## Table of Contents

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| 1.0 Introduction.....                                                          | 1  |
| 2.0 Related Works .....                                                        | 3  |
| 3.0 Methodology .....                                                          | 5  |
| 4.0 Business Understanding .....                                               | 8  |
| 5.0 Data Understanding .....                                                   | 9  |
| 5.1 Data Selection.....                                                        | 10 |
| 6.0 Data Preparation .....                                                     | 12 |
| 6.1 Missing values.....                                                        | 12 |
| 6.2 Detecting Outliers.....                                                    | 12 |
| 6.3 Feature Engineering.....                                                   | 14 |
| 6.3.1 Feature selection .....                                                  | 14 |
| 6.3.2 Normalisation.....                                                       | 17 |
| 7.0 Modelling .....                                                            | 20 |
| 7.1 Train and Test set .....                                                   | 20 |
| 7.2 Build Model.....                                                           | 20 |
| 7.2.1 Linear Regression (LR) .....                                             | 21 |
| 7.2.2 Support Vector Regression (SVR) with linear and polynomial kernels ..... | 22 |
| 7.2.3 Decision Tree (DT) .....                                                 | 23 |
| 7.2.4 Random Forest (RF) with varying numbers of trees (100, 200, 500) .....   | 24 |
| 7.2.5 K-Nearest Neighbours (KNN).....                                          | 25 |
| 7.2.6 Long Short-Term Memory (LSTM).....                                       | 26 |
| 8.0 Evaluation.....                                                            | 28 |
| 8.1 Model Performance .....                                                    | 28 |
| 8.2 Hyperparameter Tuning .....                                                | 29 |
| 9.0 Deployment .....                                                           | 32 |
| 10.0 Conclusion .....                                                          | 34 |
| References.....                                                                | 35 |

## Table of Figures

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| Figure 1 CRISP – DM phases from the CRISP-DM 1.0 Guideline .....                   | 7  |
| Figure 2 Structure of the dataset .....                                            | 10 |
| Figure 3 Dropping unnecessary columns. ....                                        | 11 |
| Figure 4 Check missing data .....                                                  | 12 |
| Figure 5 Outlier Count .....                                                       | 13 |
| Figure 6 Flagging outliers .....                                                   | 14 |
| Figure 7 R code to calculate and plot correlation matrix .....                     | 15 |
| Figure 8 Correlation matrix plot .....                                             | 15 |
| Figure 9 Using Random Forest to check feature importance.....                      | 16 |
| Figure 10 Percentage Increase in Mean Squared Error.....                           | 16 |
| Figure 11 Histogram of SALE_PRC.....                                               | 17 |
| Figure 12 Log transformation on SALE_PRC.....                                      | 18 |
| Figure 13 Normalised histogram of SALE_PRC.....                                    | 18 |
| Figure 14 Data after feature normalisation.....                                    | 19 |
| Figure 15 Splitting train and test set .....                                       | 20 |
| Figure 16 Build Linear Regression Model .....                                      | 21 |
| Figure 17 Comparison of Actual and Predicted Sale Prices (Linear Regression) ..... | 22 |
| Figure 18 Building SVR model (Linear and Poly).....                                | 23 |
| Figure 19 Building a decision tree model .....                                     | 24 |
| Figure 20 Building a Random Forest model .....                                     | 25 |
| Figure 21 Building a KNN Model.....                                                | 26 |

|                                              |    |
|----------------------------------------------|----|
| Figure 22 Building a LSTM Model .....        | 27 |
| Figure 23 Using a LSTM Model .....           | 27 |
| Figure 24 Comparing model performance .....  | 28 |
| Figure 25 Visualising model comparison ..... | 29 |
| Figure 26 Best ntree value.....              | 31 |
| Figure 27 Predicting SALE_PRC.....           | 33 |

## 1.0 Introduction

Housing prices are rising annually, there is a growing need to leverage machine learning in real estate to develop accurate price prediction models. Such models can assist developers in setting competitive selling prices and help buyers plan their budgets effectively. Numerous factors influence housing prices some positively, such as location and amenities, while others, like property age and condition, may have a negative impact (Ranjith and Ganesh, 2024). Data mining and machine learning play crucial roles in modern data analysis, with widespread applications across different sectors such as finance, healthcare, marketing, cybersecurity, real estate and many others. These techniques focus on uncovering valuable trends and knowledge from large amounts of data, empowering businesses and organisations to base their strategies on empirical evidence rather than intuition (Han, Pei and Tong, 2023).

This report will apply the CRISP-DM methodology to a housing dataset to explore the data, clean and prepare it, and use machine learning techniques to build predictive models. These predictive models would be evaluated to determine which would be best to use in predicting house prices for the dataset. The findings from this analysis will contribute to a better understanding of some determinants of housing price and offer valuable insights for stakeholders in the real estate market to predict house prices.

Data mining focuses on discovering hidden patterns, correlations, and anomalies within vast datasets using techniques such as clustering, classification and outlier detection. For example, in healthcare, data mining is used to identify disease trends and optimise treatment plans. In finance, it aids in fraud detection and risk assessment (Tan *et al.*, 2019). Machine learning leverages

algorithms and statistical models to improve system performance through experience without explicit programming. Supervised learning (regression, decision trees) and unsupervised learning (k-means clustering, principal component analysis) are widely used to build predictive models (Bishop, 2010), while Deep learning (Convolutional Neural Networks, Long Short-Term Memory Networks), is a specialised branch of machine learning that has revolutionised fields such as image recognition, computer vision, natural language processing and speech recognition by enabling machines to learn complex patterns from large amounts of data it is inspired by the human brain—to automatically detect and learn hierarchical representations of data (LeCun, Bengio and Hinton, 2015).

## 2.0 Related Works

The field of house price prediction has seen an increase in research efforts, with various machine learning models being used to improve accuracy and efficiency. According to Ritu (2015), the Random Forest model consistently outperforms other house price prediction algorithms in many studies. For example, the study by Supriya *et al.* (2023) shows that Random Forest outperforms Decision Tree and Linear Regression approaches in terms of root-mean-square error (RMSE) and variance in price prediction. A comparative study on the Chengdu housing dataset evaluates the performance of several machine learning models, including Random Forest, XGBoost, Support Vector Regression (SVR), a back-propagation neural network, and a regionally weighted regression model. Results from 5-fold cross-validation show that the Random Forest model achieves the highest accuracy, with a mean absolute error (MAE) of 1517.29 outperforming all other tested models (Li *et al.*, 2020). In another study, the random forest model demonstrated superior performance compared to deep learning. It was evaluated against alternative approaches, including neural networks, inverse distance weighting, and Kriging, using key accuracy metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and Mean Absolute Scale Error (MASE). The analysis was conducted on a real estate dataset provided by South Korea Ministry of Land, Infrastructure and Transport (MOLIT), containing property sales records from 2016 to 2019 in Seoul (Kim *et al.*, 2022).

Gao *et al.* (2022) explored property valuation models in the Greater Sydney Region, analysing how different geographical sub-areas (city-wide, broader region and finer sub-areas) influence price predictions. The findings reveal that Random Forest and Gradient Boosting outperform other methods, with higher accuracy when applied to smaller, high-resolution sub-areas. This research

demonstrates that smaller regional segmentation improves valuation accuracy and provides actionable insights into location-specific price drivers. Other research on predicting house prices found other techniques to be more accurate as different factors contribute to determine what model would be best. A paper by Ranjith and Ganesh (2024) developed a housing price prediction model trained on a 2001 dataset to forecast market prices based on key influencing factors, multiple regression methods were evaluated, with multiple linear regression selected due to its flexibility, probabilistic learning approach and high predictive performance. The model considers various determinants of housing prices, including property age, location (longitude, latitude), size (number of rooms, bathrooms, floors), nearby amenities (schools, parks, shopping centres), and overall condition.

And another study by Ibrahim *et al.* (2025) proposed an Extra Tree Regression model for house price prediction, due to its enhanced randomness during tree construction, which reduces overfitting and variance compared to traditional methods like the House Price Index (HPI) and Random Forest. The model was trained and tested on the Kaggle Boston Housing, with performance evaluated using Mean Absolute Error (MAE) and Mean Squared Error (MSE). Results demonstrated that the Extra Tree Regression outperformed the Random Forest model, achieving lower error metrics, indicating higher accuracy and reliability, making it a robust alternative to manual methods and existing machine learning approaches as it addresses vulnerabilities like human error and fraud in traditional real estate pricing.



### 3.0 Methodology

The complexity of the real estate market, influenced by multiple factors such as location, property features, economic conditions, and market trends, makes house price prediction a challenging yet crucial task. To address such intricate data mining problems, structured methodologies provide a systematic approach, ensuring comprehensive analysis and robust solutions (Mukesh *et al.*, 2025).

The Cross-Industry Standard Process for Data Mining (CRISP-DM) methodology serves as a widely adopted framework for data mining projects, offering a sequence of six interconnected phases that guide the process from initial understanding to deployment. These phases include:

- i. Business Understanding: This phase focuses on defining the project objectives and requirements, determining data mining goals and producing a project plan. A clear understanding of the business understanding phase establishes a good foundation for the entire data mining process (Supriya *et al.*, 2023).
- ii. Data Understanding: Involves the collection, description, exploring for patterns, and verification of data quality to ensure the data is appropriate for the analysis and modelling phases (Han, Pei and Tong, 2023).
- iii. Data Preparation: This phase involves selecting, cleaning, constructing, integrating and formatting the data for modelling to ensure it is suitable for analysis (James *et al.*, 2023).

- iv. Modelling: This phase focuses on constructing and refining models by selecting suitable techniques, training, tuning and assessing model performance (Han, Pei and Tong, 2023).
- v. Evaluation: This involves evaluating the model results in the context of the business understanding objectives, reviewing the overall process and making decisions about deployment (Ritu, 2015).
- vi. Deployment: The final phase where the model is implemented and made available for use, like embedding it into workflows or automating decisions. It also includes strategies for ongoing performance monitoring and model maintenance (Mukesh *et al.*, 2025).

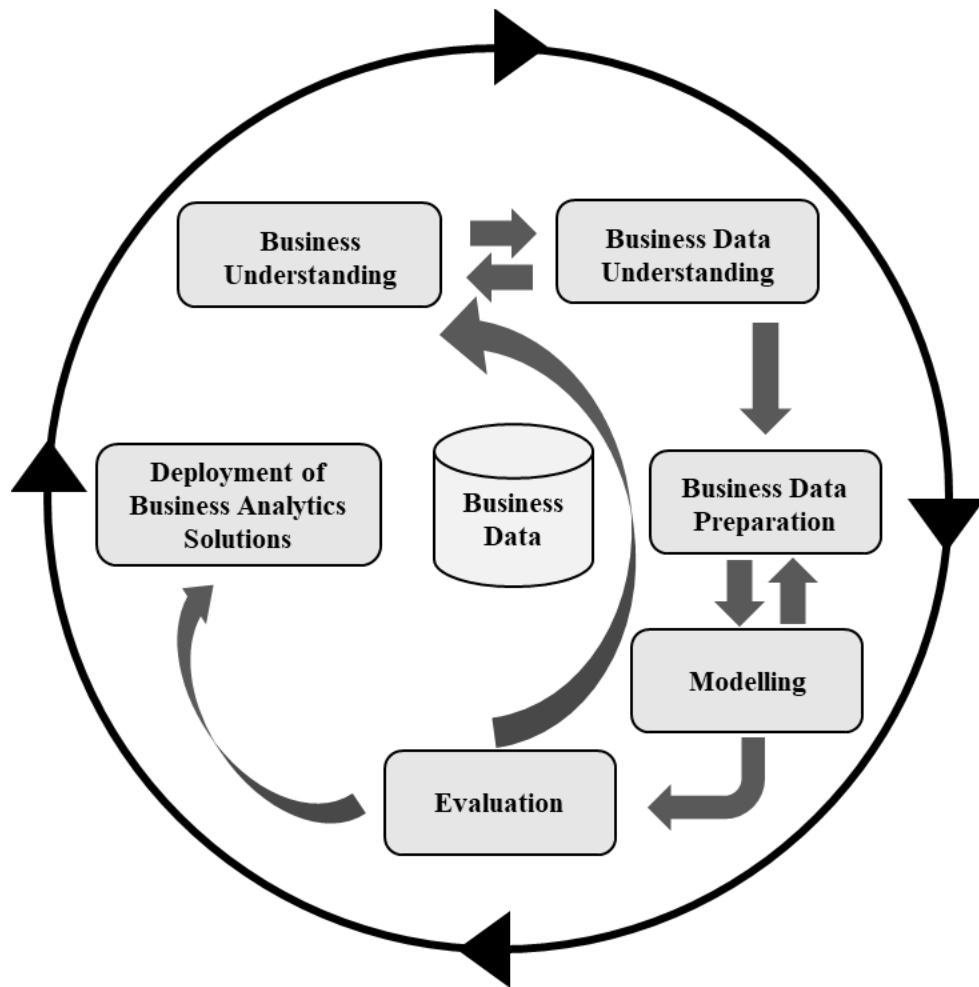


Figure 1 CRISP – DM phases from the CRISP-DM 1.0 Guideline (Chapman et al., 1999)

## 4.0 Business Understanding

Housing price prediction is essential in the real estate market, where both buyers, sellers and investors are highly dependent on accurate property price forecasts. A clear understanding of the business goals is fundamental to effectively apply data mining techniques. In this case, the primary business objective is to develop a model that can assist stakeholders in the housing market to predict housing prices based on a variety of features using data mining and machine learning techniques, in this analysis, we focus on building a model that can predict the sale price (SALE\_PRC) of houses based on various features such as land size (LND\_SQFOOT), total living area (TOT\_LVG\_AREA) and distance to the nearest highway (HWY\_DIST), among others. To achieve this, we will apply several data mining and machine learning techniques, including feature selection, data preprocessing, and model development, with a focus on predicting the target variable SALE\_PRC. The business success criteria include providing transparent insights into the features that most significantly influence the sale price, minimising the error in the predicted sale prices, and predicting prices for the following inputs.

PARCELNO: 728980145245, LND\_SQFOOT: 11247, TOTLVGAREA: 4552, SPECFEATVAL: 2105, RAIL\_DIST: 4871.9, OCEAN\_DIST: 18507.2, WATER\_DIST:375.8, CNTR\_DIST: 43897.9, SUBCNTR\_DI: 40115.7, HWY\_DIST: 41917.1, age: 42, avno60plus: 0, structure\_quality: 5, month\_sold: 8

## 5.0 Data Understanding

The dataset provided for this analysis consists of various features related to housing properties, including physical characteristics, geographical location, and sale price. The dataset includes columns such as:

- I.      PARCELNO: unique identifier for each property. About 1% appear multiple times.
- II.     SALE\_PRC: The sale price of the house (target variable).
- III.    LND\_SQFOOT: The size of the land in square feet.
- IV.    TOT\_LVG\_AREA: The total living area of the house in square feet.
- V.     SPEC\_FEAT\_VAL: The presence of special features in the house, like a pool or garden.
- VI.    RAIL\_DIST: Distance to the nearest railroad (feet).
- VII.   OCEAN\_DIST: Distance to the nearest ocean (feet).
- VIII.  WATER\_DIST: Distance to the nearest body of water (feet).
- IX.    CNTR\_DIST: Distance to the central business district (feet).
- X.     SUBCNTR\_DI: Distance to the nearest subcentre (feet).
- XI.    HWY\_DIST: Distance to the nearest highway (an indicator of noise) (feet).
- XII.   Age: age of the structure.
- XIII.  avno60plus: Dummy variable for airplane noise exceeding an acceptable level
- XIV.   structure quality: Quality of the structure
- XV.    month sold: Month the house was sold
- XVI.   LATITUDE: Measure of location
- XVII.  LONGITUDE: Measure of location

The dataset contains 13932 observations (rows) of 17 variables (columns) and all the variables were numerical

```
{r}
# Load the data
Housing <- read.csv("Housing Data_Same Region.csv")

# check the structure of the data
str(Housing)
```

'data.frame': 13932 obs. of 17 variables:

|                  |       |                                                            |
|------------------|-------|------------------------------------------------------------|
| \$ LATITUDE      | : num | 25.9 25.9 25.9 25.9 25.9 ...                               |
| \$ LONGITUDE     | : num | -80.2 -80.2 -80.2 -80.2 -80.2 ...                          |
| \$ PARCELNO      | : num | 6.22e+11 6.22e+11 6.22e+11 6.22e+11 6.22e+11 ...           |
| \$ SALE_PRC      | : num | 440000 349000 800000 988000 755000 630000 1020000          |
| \$ LND_SQFOOT    | : int | 9375 9375 9375 12450 12800 9900 10387 10272 9375 13803 ... |
| \$ TOT_LVG_AREA  | : int | 1753 1715 2276 2058 1684 1531 1753 1663 1493 3077 ...      |
| \$ SPEC_FEAT_VAL | : int | 0 0 49206 10033 16681 2978 23116 34933 11668 34580 ...     |
| \$ RAIL_DIST     | : num | 2816 4359 4413 4585 4063 ...                               |
| \$ OCEAN_DIST    | : num | 12811 10648 10574 10156 10837 ...                          |
| \$ WATER_DIST    | : num | 348 338 297 0 327 ...                                      |
| \$ CNTR_DIST     | : num | 42815 43505 43530 43798 43600 ...                          |
| \$ SUBCNTR_DI    | : num | 37742 37341 37329 37423 37551 ...                          |
| \$ HWY_DIST      | : num | 15955 18125 18201 18514 17903 ...                          |
| \$ age           | : int | 67 63 61 63 42 41 63 21 56 63 ...                          |
| \$ avno60plus    | : int | 0 0 0 0 0 0 0 0 0 ...                                      |
| \$ month sold    | : int | 8 9 2 9 7 2 2 9 3 11 ...                                   |

*Figure 2 Structure of the dataset*

## 5.1 Data Selection

The dataset is labelled as "Same Region" which implies minimal geographical variation, unnecessary columns like LATITUDE and LONGITUDE were removed as they provide geographical data that is irrelevant to the analysis. PARCELNO was also removed as it serves as a unique identifier.

```
# Drop columns that are not needed  
HouseData <- Housing |> select(-LATITUDE, -LONGITUDE, -PARCELNO)
```

*Figure 3 Dropping unnecessary columns.*

## 6.0 Data Preparation

### 6.1 Missing values

Missing values mean that some information is absent in the dataset, handling missing data is a critical step in data preprocessing and different strategies can be used, depending on the nature of the data and the reason for missing values (Little and Rubin, 2020). Checking for missing values in the dataset shows that it did not contain missing values and as such they did not need to be handled.

```
# Check for missing data
colSums(is.na(HouseData))
```

| LND_SQFOOT        | TOT_LVG_AREA | SPEC_FEAT_VAL | RAIL_DIST |
|-------------------|--------------|---------------|-----------|
| OCEAN_DIST        | WATER_DIST   |               |           |
| 0                 | 0            | 0             | 0         |
| CNTR_DIST         | SUBCNTR_DI   | HWY_DIST      | age       |
| avno60plus        | month_sold   |               |           |
| 0                 | 0            | 0             | 0         |
| structure_quality | SALE_PRC     |               |           |
| 0                 | 0            |               |           |

Figure 4 Check missing data

### 6.2 Detecting Outliers

Outliers are data points that are distant from other data points in the dataset. They can occur for various reasons, such as measurement errors, data entry errors, or natural variability (Aggarwal, 2013). Outliers were identified using the Interquartile Range (IQR) method, which flagged extreme values in the numerical columns. This step is crucial as outliers can distort the results of certain machine learning models.



| VARIABLES         | OUTLIER COUNT |
|-------------------|---------------|
| LND_SQFOOT        | 1370          |
| TOT_LVG_AREA      | 447           |
| SPEC_FEAT_VAL     | 1703          |
| RAIL_DIST         | 137           |
| OCEAN_DIST        | 0             |
| WATER_DIST        | 245           |
| CNTR_DIST         | 3             |
| SUBCNTR_DI        | 120           |
| HWY_DIST          | 291           |
| AGE               | 3             |
| AVNO60PLUS        | 208           |
| MONTH_SOLD        | 0             |
| STRUCTURE_QUALITY | 0             |
| SALE_PRC          | 1340          |

*Figure 5 Outlier Count*

By identifying and handling outliers (either by removing, transforming, or treating them separately), the stability of models can be improved, especially those that rely on the distribution of data.

To evaluate the outliers, we flagged the outliers by creating new variables that represent each original variable where outliers were detected and added these flags as new columns in the dataset.

|   | structure_quality | SALE_PRC | LND_SQFOOT_outlier | TOT_LVG_AREA_outlier | SPEC_FEAT_VAL_outlier | RAIL_DIST_outlier | OCEAN_DIST_outlier | WATER_DIST_outlier |
|---|-------------------|----------|--------------------|----------------------|-----------------------|-------------------|--------------------|--------------------|
| 5 | 5                 | 1067300  | 0                  | 0                    | 0                     | 0                 | 0                  |                    |
| 2 | 5                 | 1056000  | 0                  | 0                    | 1                     | 0                 | 0                  |                    |
| 9 | 5                 | 900000   | 1                  | 1                    | 0                     | 0                 | 0                  |                    |
| 5 | 5                 | 1015000  | 0                  | 0                    | 1                     | 0                 | 0                  |                    |
| 8 | 5                 | 1235000  | 0                  | 0                    | 1                     | 0                 | 0                  |                    |
| 6 | 5                 | 1587500  | 0                  | 1                    | 1                     | 0                 | 0                  |                    |
| 1 | 5                 | 1265000  | 0                  | 0                    | 0                     | 0                 | 0                  |                    |
| 1 | 5                 | 2450000  | 0                  | 1                    | 1                     | 0                 | 0                  |                    |
| 6 | 5                 | 980000   | 0                  | 0                    | 1                     | 0                 | 0                  |                    |
| 8 | 5                 | 1130000  | 0                  | 0                    | 0                     | 0                 | 0                  |                    |
| 2 | 5                 | 1340000  | 0                  | 0                    | 1                     | 0                 | 0                  |                    |

*Figure 6 Flagging outliers*

Outliers can represent rare but important cases that may require further investigation. For example, a data point representing a luxury house may be an outlier in a dataset of regular houses, but it may provide important insight into market dynamics. As in the dataset used in this, the outliers detected was in cases that indicated the dynamics of housing prices and did not need to be handled.

## 6.3 Feature Engineering

### 6.3.1 Feature selection

Feature selection was based on correlation analysis and feature importance obtained from the Random Forest model. Variables like avno60plus and month\_sold showed minimal impact on the target variable and were removed to simplify the model.

# Feature Engineering

```
{r}  
# Calculate the correlation matrix  
corMatrix <- cor(HouseData[, sapply(HouseData, is.numeric)])  
print(corMatrix)  
  
# Plot the correlation matrix  
corrplot(corMatrix, method = "color", type = "lower", order = "hclust",  
          tl.cex = 0.8, # Text label size  
          number.cex = 0.6, # Number size in cells  
          col = colorRampPalette(c("blue", "white", "red"))(200))
```

Figure 7 R code to calculate and plot correlation matrix

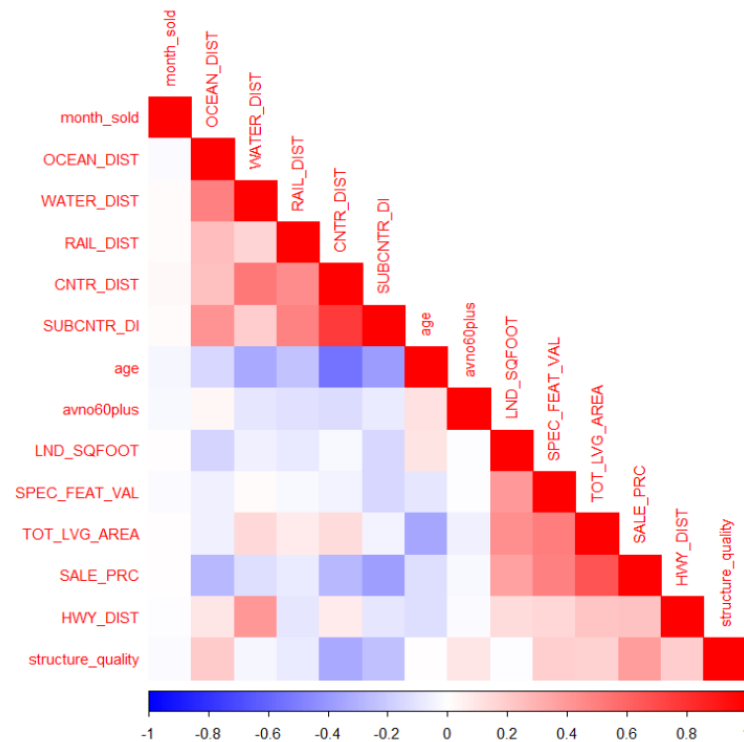


Figure 8 Correlation matrix plot

```
# Using rf model to check feature importance
checkImp <- randomForest(SALE_PRC ~ ., data = HouseData, importance = TRUE)
print(importance(checkImp))

# Extract importance as a data frame
imp_data <- as.data.frame(importance(checkImp))
imp_data$Variable <- rownames(imp_data) # Add variable names as a column

# Plot %IncMSE using ggplot2
ggplot(imp_data, aes(x = reorder(Variable, `%IncMSE`), y = `%IncMSE`)) +
  geom_bar(stat = "identity", fill = "lightblue") +
  coord_flip() + # Flip for horizontal bars
  labs(title = "Variable Importance (%IncMSE) for SALE_PRC Prediction",
       x = "Variables", y = "Increase in MSE (%)") +
  theme_minimal()
```

Figure 9 Using Random Forest to check feature importance

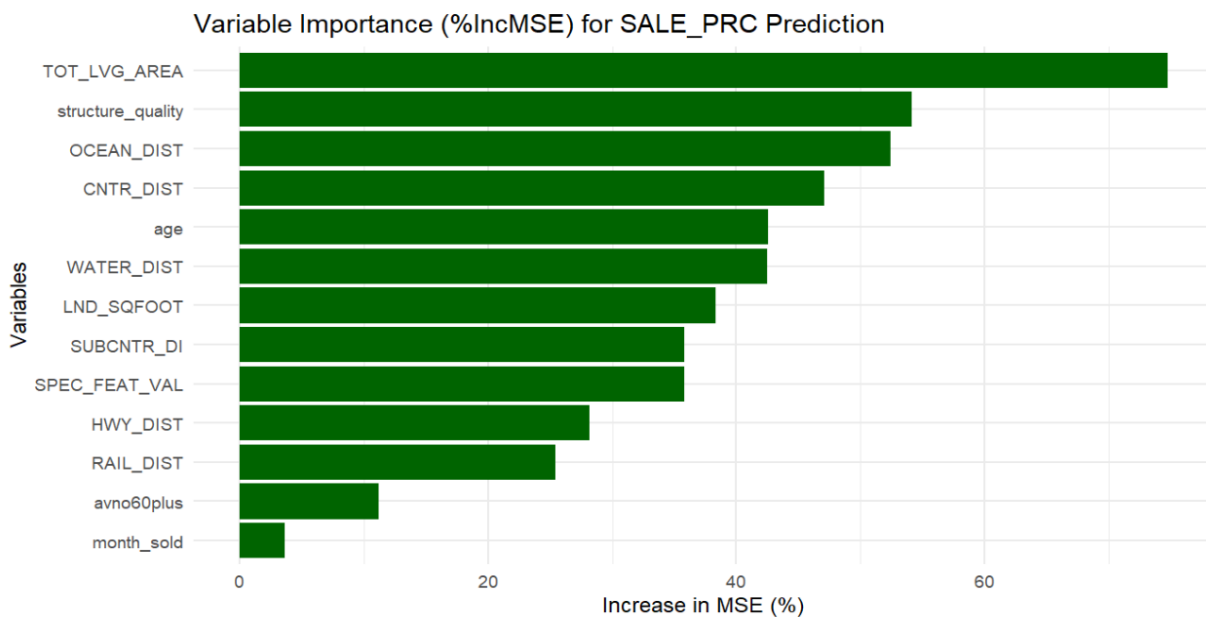


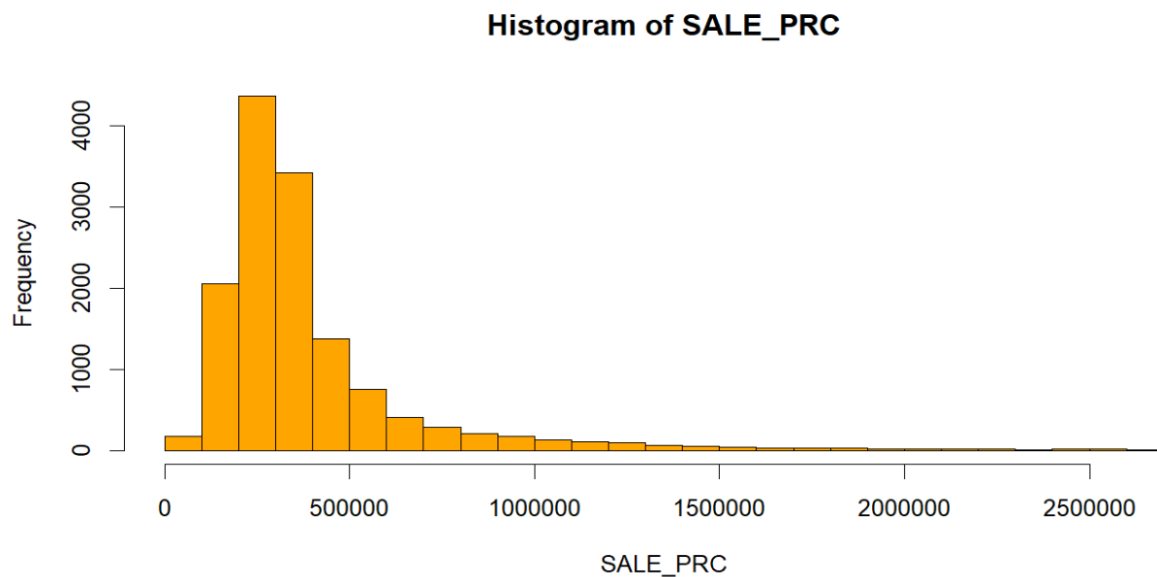
Figure 10 Percentage Increase in Mean Squared Error

Based on the results of the correlation matrix, percentage increase in mean squared error (%IncMSE) and increase in node purity (IncNodePurity), columns avno60plus and month\_sold are non-important and should be removed to simplify the model.

### 6.3.2 Normalisation

Normalisation ensures fair contribution from all features, improves model performance, and accelerates training. Normalisation is a critical preprocessing step in machine learning to ensure features are on a comparable scale as any algorithms such as SVMs, k-NN and neural networks rely on distance or weighted feature combinations (Aggarwal, 2013).

Plotting a histogram for the target variable would show that it is not normally distributed and to mitigate this and enhance the model performance involved performing log transformation to transform the data, log transformation addresses skewed distributions making the data more symmetrical or closer to a normal distribution.



*Figure 11 Histogram of SALE\_PRC*

```
# Perform log transformation on the SALE_PRC column
HouseData_scale$SALE_PRC <- log(HouseData$SALE_PRC)

hist(HouseData_scale$SALE_PRC,
     breaks = 50,                    # Adjust number of bins
     main = "Histogram of Sale Price", # Title of the histogram
     xlab = "Sale Price",             # X-axis label
     col = "orange",                  # Bar color
     border = "black")
```

Figure 12 Log transformation on SALE\_PRC

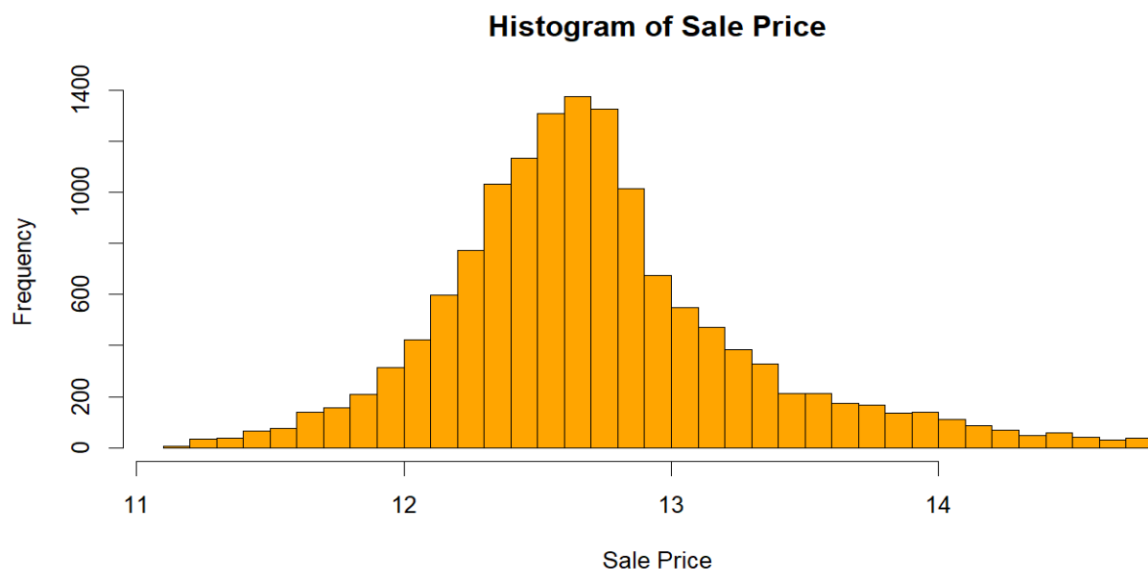


Figure 13 Normalised histogram of SALE\_PRC

The other variables were scaled using `preProcess()` from the `caret` package.

```
# scale the other vaariables to normalise data
HouseData_norm <- preProcess(HouseData_scale[, -which(names(HouseData_scale) ==
"SALE_PRC")], method = "range")

# Apply the normalisation model to the data (excluding SALE_PRC column)
HouseData_scale[, -which(names(HouseData_scale) == "SALE_PRC")] <- predict
(HouseData_norm, HouseData_scale[, -which(names(HouseData_scale) == "SALE_PRC")])
View(HouseData_scale)
```

Figure 14 Scaling predicting variables

| RAIL_DIST   | OCEAN_DIST  | WATER_DIST   | CNTR_DIST | SUBCNTR_DI | HWY_DIST  | age        | avno60plus | month_sold | structure_quality | SALE_PRC |
|-------------|-------------|--------------|-----------|------------|-----------|------------|------------|------------|-------------------|----------|
| 0.094741819 | 0.166540854 | 6.896853e-03 | 0.2496924 | 0.3325609  | 0.3299845 | 0.69791667 | 0          | 0.63636364 | 0.75              | 12.99453 |
| 0.146857587 | 0.137895186 | 6.702408e-03 | 0.2541087 | 0.3288786  | 0.3751225 | 0.65625000 | 0          | 0.72727273 | 0.75              | 12.76283 |
| 0.148674479 | 0.136911194 | 5.894865e-03 | 0.2542720 | 0.3287705  | 0.3766929 | 0.63541667 | 0          | 0.09090909 | 0.75              | 13.59237 |
| 0.154486508 | 0.131380713 | 0.000000e+00 | 0.2559825 | 0.3296367  | 0.3832219 | 0.65625000 | 0          | 0.72727273 | 0.75              | 13.80344 |
| 0.136871433 | 0.140390259 | 6.480184e-03 | 0.2547158 | 0.3308064  | 0.3705132 | 0.43750000 | 0          | 0.54545455 | 0.75              | 13.53447 |
| 0.080405930 | 0.169263715 | 3.748031e-03 | 0.2517405 | 0.3365392  | 0.3244164 | 0.42708333 | 0          | 0.09090909 | 0.75              | 13.35348 |
| 0.110327243 | 0.151395599 | 0.000000e+00 | 0.2547094 | 0.3346848  | 0.3531411 | 0.65625000 | 0          | 0.09090909 | 1.00              | 13.83531 |
| 0.104754990 | 0.152061746 | 2.083342e-04 | 0.2558756 | 0.3367418  | 0.3515125 | 0.21875000 | 0          | 0.72727273 | 0.75              | 13.65299 |
| 0.069950356 | 0.169618640 | 1.021829e-03 | 0.2539601 | 0.3398924  | 0.3230873 | 0.58333333 | 0          | 0.18181818 | 0.75              | 12.42922 |
| 0.098855155 | 0.154705147 | 1.924611e-04 | 0.2555509 | 0.3370801  | 0.3230873 | 0.65625000 | 0          | 0.90909091 | 1.00              | 14.01436 |
| 0.118249975 | 0.143376666 | 2.400803e-04 | 0.2573997 | 0.3361350  | 0.3645831 | 0.66666667 | 0          | 0.09090909 | 0.75              | 13.96393 |
| 0.007845058 | 0.199636334 | 1.465482e-02 | 0.2516918 | 0.3471469  | 0.2775417 | 0.53125000 | 0          | 0.90909091 | 0.25              | 12.46844 |
| 0.021671001 | 0.192993399 | 7.271854e-03 | 0.2524417 | 0.3459259  | 0.2878044 | 0.58333333 | 0          | 0.54545455 | 0.25              | 12.08954 |
| 0.023454122 | 0.192214682 | 7.007964e-03 | 0.2525480 | 0.3457847  | 0.2892229 | 0.68750000 | 0          | 0.54545455 | 0.25              | 12.42922 |
| 0.081797305 | 0.158850359 | 0.000000e+00 | 0.2575848 | 0.3415928  | 0.3411978 | 0.47916667 | 0          | 0.72727273 | 1.00              | 13.72504 |
| 0.005183884 | 0.197955735 | 1.066671e-02 | 0.2538717 | 0.3496760  | 0.2800834 | 0.67708333 | 0          | 0.90909091 | 0.25              | 11.97666 |
| 0.006717098 | 0.197102854 | 9.388926e-03 | 0.2539576 | 0.3495155  | 0.2813169 | 0.50000000 | 0          | 0.45454545 | 0.25              | 12.20106 |
| 0.062041133 | 0.167330165 | 8.333366e-04 | 0.2573248 | 0.3440394  | 0.3264777 | 0.64583333 | 0          | 0.81818182 | 0.75              | 13.28788 |

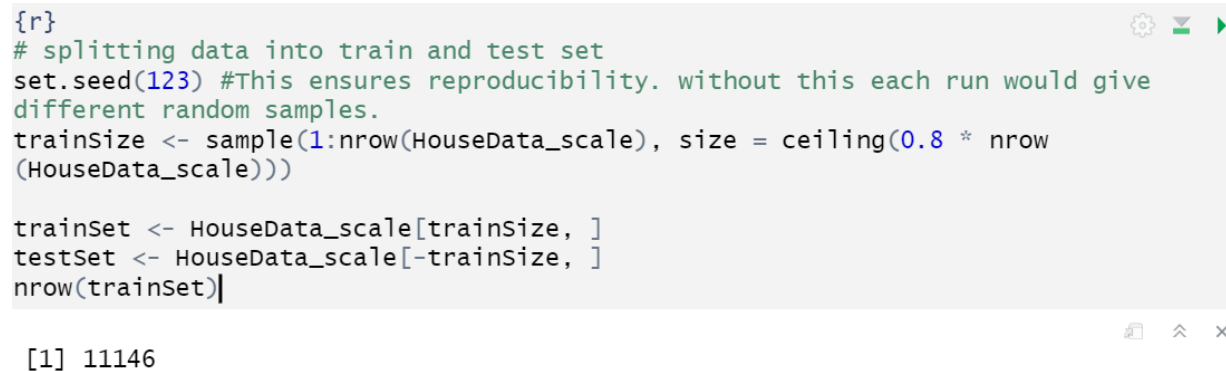
Figure 15 Data after feature normalisation

## 7.0 Modelling

Modelling involves selecting and applying various modelling techniques, building the models, and assessing their performance.

### 7.1 Train and Test set

Each model was trained on the training set (80% of the data) and tested on the test set (20% of the data) using `set.seed(123)` to ensure reproducibility.



```
{r}
# splitting data into train and test set
set.seed(123) #This ensures reproducibility. without this each run would give
different random samples.
trainSize <- sample(1:nrow(HouseData_scale), size = ceiling(0.8 * nrow
(HouseData_scale)))

trainSet <- HouseData_scale[trainSize, ]
testSet <- HouseData_scale[-trainSize, ]
nrow(trainSet)
```

[1] 11146

*Figure 16 Splitting train and test set*

And after splitting, the training set had 11146 observations while the test set had 2786 observations.

### 7.2 Build Model

Various machine learning algorithms were applied to predict house prices. The models selected for this analysis were:



### 7.2.1 Linear Regression (LR)

Linear regression is one of the simplest and most widely used statistical methods for predicting a continuous dependent variable based on one or more independent variables. It assumes a linear relationship between the input variables and the target variable and is computationally inexpensive, making it suitable for large datasets, the model is sensitive to outliers which can distort the regression line and affect predictions (James *et al.*, 2023). Linear Regression was implemented using the `lm()` function, fitting a linear relationship between the predictor variables and the log-transformed sale price.

## train a linear regression model

```
{r}
lm_model <- lm(SALE_PRC ~ ., data = trainSet)

# Save the trained model to a file
saveRDS(lm_model, "lm_model.rds")

# Make predictions on the test set
lmPredict <- predict(lm_model, newdata = testSet)

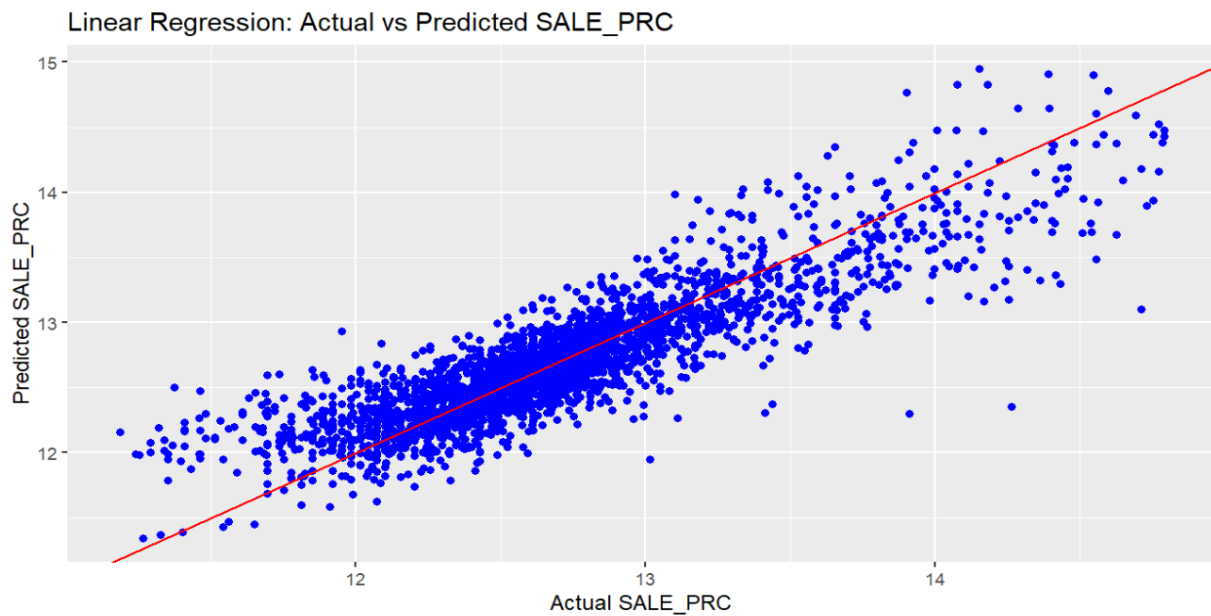
# check model performance
lm_perform0 <- postResample(pred = lmPredict, obs = testSet$SALE_PRC)
print(lm_perform)

# save MAE to use for comparison
LR_MAE <- mae(testSet$SALE_PRC, lmPredict)

#visualise the predicted vs actual values
ggplot(data = data.frame(Actual = testSet$SALE_PRC, Predicted = lmPredict), aes(x =
Actual, y = Predicted)) +
  geom_point(color = 'blue') +
  geom_abline(slope = 1, intercept = 0, color = 'red') +
  xlab("Actual SALE_PRC") +
  ylab("Predicted SALE_PRC") +
  ggtitle("Linear Regression: Actual vs Predicted SALE_PRC")
```

Figure 17 Build Linear Regression Model

The scatter plot below shows the comparison between the actual sale prices and the predicted sale prices from the linear regression model. The red line represents the ideal scenario where the predicted sale prices perfectly match the actual values. Points scattered around the line indicate the degree of deviation from the predicted values, which helps to evaluate the model accuracy.



*Figure 18 Comparison of Actual and Predicted Sale Prices (Linear Regression)*

### 7.2.2 Support Vector Regression (SVR) with linear and polynomial kernels

Support Vector Regression (SVR) is an extension of Support Vector Machines (SVM), which can be used for both classification and regression tasks. The linear kernel in SVR performs well when the data exhibits a linear relationship between the predictors and the target variable while the polynomial kernel allows SVR to handle non-linear relationships by mapping the data into a higher-dimensional space (Hastie, Tibshirani and Friedman, 2011). Support Vector Regression was

implemented with both a linear and a polynomial kernel using the `svm()` function, allowing for the exploration of different types of decision boundaries.

## Train the SVR model

```
{r}
# Train the SVR model for linear kernel to predict SALE_PRC
svrModel <- svm(SALE_PRC ~ ., data = trainSet, kernel = "linear")
svrLinPredict <- predict(svrModel, newdata = testSet)

# Save the linear kernel SVR model
saveRDS(svrModel, "svr_linear_model.rds")

#calculating the performance metrics
SVRPerform <- postResample(pred = svrLinPredict, obs = testSet$SALE_PRC)

SVRLIN_MAE <- mae(testSet$SALE_PRC, svrLinPredict)

# Train the SVR model for poly kernel to predict SALE_PRC
svrModel2 <- svm(SALE_PRC ~ ., data = trainSet, kernel = "poly")
svrPolyPredict <- predict(svrModel2, newdata = testSet)

# Save the polynomial kernel SVR model
saveRDS(svrModel2, "svr_poly_model.rds")

SVRPerform2 <- postResample(pred = svrPolyPredict, obs = testSet$SALE_PRC)
SVRPOLY_MAE <- mae(testSet$SALE_PRC, svrPolyPredict)
```

*Figure 19 Building SVR model (Linear and Poly)*

### 7.2.3 Decision Tree (DT)

Decision trees are supervised learning models that make predictions by recursively splitting the data into subsets based on the values of input features. At each node, the algorithm chooses the feature that best splits the data, and the process continues until it reaches a predefined stopping condition (Krzywinski and Altman, 2017).

# Train a Decision tree model

```
{r}

dtModel <- rpart(SALE_PRC ~ ., data = trainSet)
dtPredict <- predict(dtModel, newdata = testSet)

# Save the Decision Tree model
saveRDS(dtModel, "dt_model.rds")

dtPerform <- postResample(pred = dtPredict, obs = testSet$SALE_PRC)
DT_MAE <- mae(testSet$SALE_PRC, dtPredict)
dtPerform
DT_MAE
```

*Figure 20 Building a decision tree model*

## 7.2.4 Random Forest (RF) with varying numbers of trees (100, 200, 500)

Random Forest is a learning method that combines multiple decision trees to produce a more accurate and robust model. The model works by aggregating the predictions from several decision trees

# Train a RandomForest Model

```
{r}
# Train a RandomForest Model with ntree = 100
rfModel100 <- randomForest(SALE_PRC ~ ., data = trainSet, ntree = 100)
rfPredict100 <- predict(rfModel100, newdata = testSet)

# Save the Random Forest model with ntree = 100
saveRDS(rfModel100, "rf_model_100.rds")

rfPerform100 <- postResample(pred = rfPredict100, obs = testSet$SALE_PRC)
rfPerform100

RF100_MAE <- mae(testSet$SALE_PRC, rfPredict100)
RF100_MAE

# Train a RandomForest Model with ntree = 200
rfModel200 <- randomForest(SALE_PRC ~ ., data = trainSet, ntree = 200)
rfPredict200 <- predict(rfModel200, newdata = testSet)
rfPerform200 <- postResample(pred = rfPredict200, obs = testSet$SALE_PRC)
rfPerform200
RF200_MAE <- mae(testSet$SALE_PRC, rfPredict200)

# Save the Random Forest model with ntree = 200
saveRDS(rfModel200, "rf_model_200.rds")
```

*Figure 21 Building a Random Forest model*

## 7.2.5 K-Nearest Neighbours (KNN)

K-Nearest Neighbour is a simple, instance-based learning algorithm that predicts the target variable by considering the majority vote or average of the nearest data points. A KNN model was trained using the `caret::train()` function.

# Train a KNN model

```
{r}

knnModel <- caret::train(SALE_PRC ~ ., data = trainSet, method = "knn",
  tuneGrid = data.frame(k = 5))
knnPredict <- predict(knnModel, testSet)
knnPerform <- postResample(pred = knnPredict, obs = testSet$SALE_PRC)
print(knnPerform)
KNN_MAE <- mae(testSet$SALE_PRC, knnPredict)

# Save the trained KNN model
saveRDS(knnModel, "knn_model.rds")
```

*Figure 22 Building a KNN Model*

## 7.2.6 Long Short-Term Memory (LSTM)

To prepare the data for the LSTM model, the first step is to separate the target variable, SALE\_PRC from the features. This is done by extracting the features into the variable xTrain and xTest for the training and test datasets, respectively, while the target variable SALE\_PRC is stored in yTrain and yTest. The features are converted into matrices to make them compatible with the LSTM input format.

## Train a LSTM model

```
{r}
# Prepare data for LSTM
xTrain <- as.matrix(trainSet[, -which(names(trainSet) == "SALE_PRC")])
yTrain <- trainSet$SALE_PRC
xTest <- as.matrix(testSet[, -which(names(testSet) == "SALE_PRC")])
yTest <- testSet$SALE_PRC

# Reshaping for LSTM input
xTrain <- array_reshape(xTrain, c(nrow(xTrain), 1, ncol(xTrain)))
xTest <- array_reshape(xTest, c(nrow(xTest), 1, ncol(xTest)))

# Define the LSTM model
lstmModel <- keras_model_sequential() %>%
  layer_lstm(units = 50, input_shape = c(1, dim(xTrain)[3]), return_sequences =
FALSE) %>%
  layer_dense(units = 1)

# Compile the model
lstmModel %>% compile(
  loss = 'mean_squared_error',
  optimizer = optimizer_adam(),
  metrics = c('mean_absolute_error')
)
```

Figure 23 Building a LSTM Model

After defining the model, it is compiled using the mean squared error loss function, the LSTM model is then trained using the `fit()` function, used to make predictions and the model is saved

```
# Fit the model
lstmModel %>% fit(xTrain, yTrain, epochs = 10, batch_size = 32)

# Make predictions
lstmPredict <- lstmModel %>% predict(xTest)

# Evaluate the model
lstmPerform <- postResample(pred = lstmPredict, obs = testSet$SALE_PRC)
lstmPerform
LSTM_MAE <- mae(testSet$log_SALE_PRC, lstmPredict)

# Save the LSTM model
save_model_hdf5(lstmModel, "lstm_model.h5")
```

Figure 24 Using a LSTM Model

## 8.0 Evaluation

Model evaluation was primarily conducted using the `postResample()` function from the `caret` package and the `mae()` function from the `Metrics` package, focusing on the Mean Absolute Error (MAE) as the primary performance metric.

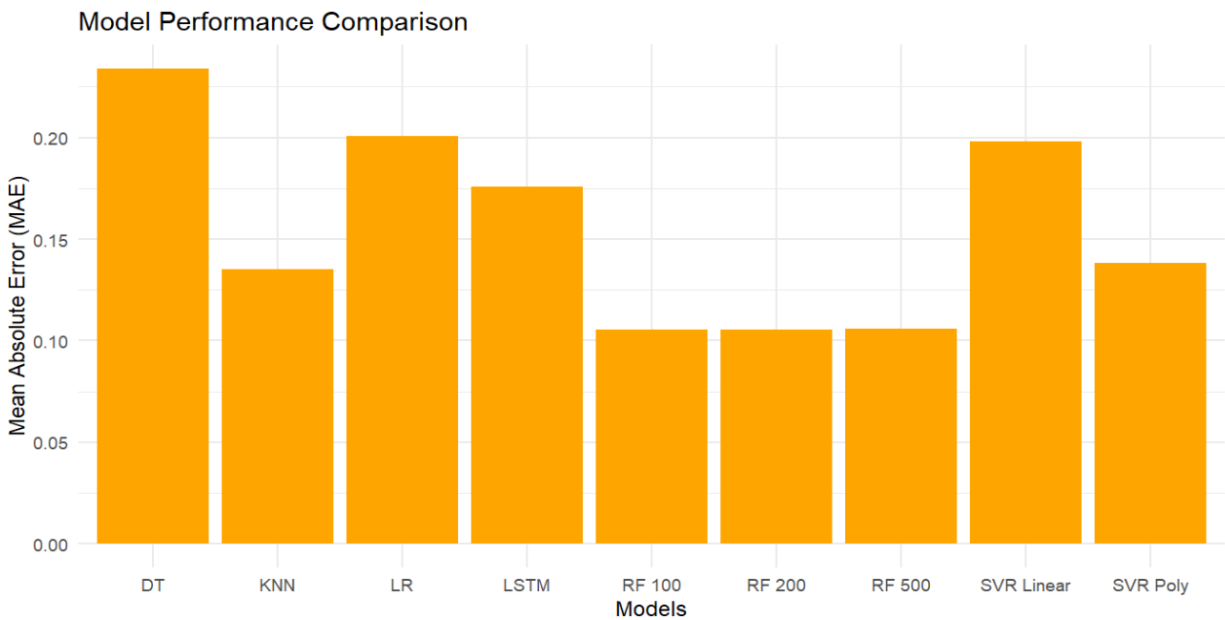
### 8.1 Model Performance

| Model                     | MAE (Log Price) | RMSE (Log Price) | R-squared |
|---------------------------|-----------------|------------------|-----------|
| Linear Regression         | 0.2008250       | 0.2746476        | 0.7663112 |
| SVR (Linear)              | 0.1978983       | 0.2794772        | 0.7604635 |
| SVR (Poly)                | 0.1383349       | 0.2062593        | 0.8693776 |
| Decision Tree             | 0.2340476       | 0.3156991        | 0.6911450 |
| Random Forest (100 Trees) | 0.1054852       | 0.1618421        | 0.9214168 |
| Random Forest (200 Trees) | 0.1052177       | 0.1614715        | 0.9217812 |
| Random Forest (500 Trees) | 0.1047415       | 0.1610843        | 0.9222269 |
| KNN                       | 0.1351777       | 0.2015112        | 0.8754257 |
| LSTM                      | 0.1759068       | 0.2395100        | 0.8249329 |

*Figure 25 Comparing model performance*



As shown in Figure 24, the Random Forest (500 Trees) model achieved the lowest MAE, the best RMSE, and R-squared value, making it the best performing model for predicting housing prices in this dataset. The SVR (Polynomial Kernel) and KNN also performed reasonably well, the LSTM model, while powerful did not outperform the simpler models in this case.



*Figure 26 Visualising model comparison*

## 8.2 Hyperparameter Tuning

Hyperparameter tuning refers to the process of finding the best set of parameters such as the number of trees in a Random Forest model, or the learning rate in a neural network that result in the best-performing model. Tuning hyperparameters is a crucial step in optimising the model performance and can significantly impact the accuracy and robustness of the final predictions

(Géron, 2023). In this project, we performed hyperparameter tuning on the Random Forest model, focusing on two primary parameters:

- i. Number of Trees (ntree): This defines how many decision trees the model should construct. A higher number of trees generally leads to more robust predictions, but it also increases computational cost.
- ii. mtry: This is the number of variables (features) to consider for each split in each decision tree. This value was fixed at 3 during the tuning process.

And we perform the following steps

- i. Define the Range for ntree: A sequence of values for the number of trees (ntree) is created, ranging from 100 to 600 with a step of 1.
- ii. Set Up the Tuning Grid: A tuning grid is defined with mtry fixed at 3, which represents the number of variables considered for each tree split.
- iii. Set Up Cross-Validation: A 5-fold cross-validation is used to evaluate the model's performance by splitting the data into 5 subsets.
- iv. Train the Model for Each ntree Value: The Random Forest model is trained for each value of ntree, and the performance is evaluated based on RMSE from cross-validation.
- v. Track the Best Model: The model with the lowest RMSE is selected as the best model, and its corresponding ntree value is stored.
- vi. Save the Best Model: The best model is saved for future use and predictions.
- vii. Make Predictions: The best model is used to make predictions on the test set.

- viii. Evaluate the Model: The performance of the best model is evaluated using MAE, RMSE, and R-squared metrics.

The best ntree value for random forest was determined to be 490

```
if(current_rmse2 < best_rmse2) {  
  best_rmse2 <- current_rmse2  
  best_model2 <- rf_model2  
  best_ntree2 <- current_ntree2 # Track the best ntree value  
}  
}  
  
# Save the best model  
saveRDS(best_model2, "best_rf_model2.rds")  
  
# Print the best ntree value  
cat("Best ntree value:", best_ntree2, "\n")
```

R RStudio: Notebook Output

Best ntree value: 490

*Figure 27 Best ntree value*

## 9.0 Deployment

The trained model can be integrated into a web application or software tool, allowing stakeholders to input details of house properties and get predicted sale prices. Continuous monitoring of the model performance is necessary to ensure its accuracy as market conditions change and the model should be retrained periodically to ensure it adapts to evolving market trends.

For this report we get a prediction for the following inputs using the optimal model.

PARCELNO: 728980145245, LND\_SQFOOT: 11247, TOTLVGAREA: 4552, SPECFEATVAL: 2105,  
RAIL\_DIST: 4871.9, OCEAN\_DIST: 18507.2, WATER\_DIST:375.8, CNTR\_DIST: 43897.9,  
SUBCNTR\_DI: 40115.7, HWY\_DIST: 41917.1, age: 42, avno60plus: 0, structure\_quality: 5,  
month\_sold: 8

The predicted sale price with the inputs above was: 1019887

```
{r}

# Load best Random Forest model
best_rf_model <- readRDS("best_rf_model2.rds")

# Prepare the new input data
new_data <- data.frame(
  LND_SQFOOT = 11247,
  TOT_LVG_AREA = 4552,
  SPEC_FEAT_VAL = 2105,
  RAIL_DIST = 4871.9,
  OCEAN_DIST = 18507.2,
  WATER_DIST = 375.8,
  CNTR_DIST = 43897.9,
  SUBCNTR_DI = 40115.7,
  HWY_DIST = 41917.1,
  age = 42,
  avno60plus = 0,
  structure_quality = 5,
  month_sold = 8
)

# Make predictions using the optimal model
rf_prediction <- predict(best_rf_model, newdata = new_data)

cat("Predicted SALE_PRC:", rf_prediction, "\n")
```

Predicted SALE\_PRC: 1019887

Figure 28 Predicting SALE\_PRC

## 10.0 Conclusion

This report explored the application of machine learning techniques to predict housing prices, using the CRISP-DM methodology. The goal was to develop a model that could help stakeholders in the real estate market predict sale prices effectively and efficiently. Various machine learning models were evaluated, including Linear Regression, Support Vector Regression, Decision Trees, Random Forests, K-Nearest Neighbours, and Long Short-Term Memory networks.

The Random Forest (500 Trees) model emerged as the best performing model, achieving the lowest MAE, the best RMSE value, Hyperparameter tuning was performed to optimise the Random Forest model by selecting the best number of trees (ntree) and the number of features to consider at each split (mtry). The optimal ntree value was found to be 490, which resulted in improved model performance.

For practical deployment, the model can be integrated into a web application or software tool, allowing real estate developers, buyers, and investors to input property features and receive predicted sale prices. Continuous monitoring and periodic retraining of the model are essential to maintain its accuracy as market conditions evolve.

## References

- Aggarwal, C.C. (2013) *Outlier analysis*. 1st ed. [Online] New York ; Heidelberg ; Dordrecht ; London: Springer. Available from: <https://> . [4/13/2025].
- Bishop, C.M. (2010) *Pattern recognition and machine learning*. 10. (corr. printing) ed. [Online] New York [u.a.]: Springer. Available from: <https://> . [4/13/2025].
- Gao, Q., Shi, V., Pettit, C. and Han, H. (2022) Property valuation using machine learning algorithms on statistical areas in Greater Sydney, Australia. *Land use policy*, [Online] 123 , pp. 106409 Available from: <https://dx.doi.org/10.1016/j.landusepol.2022.106409>. [4/12/2025].
- Géron, A. (2023) *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. Third edition, second release ed. [Online] Beijing ; Boston ; Farnham ; Sebastopol ; Tokyo: O'Reilly. Available from: [http://bvbr.bib-bvb.de:8991/F?func=service&doc\\_library=BVB01&local\\_base=BVB01&doc\\_number=034237700&sequence=000001&line\\_number=0001&func\\_code=DB\\_RECORDS&service\\_type=MEDIA](http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&local_base=BVB01&doc_number=034237700&sequence=000001&line_number=0001&func_code=DB_RECORDS&service_type=MEDIA). [4/15/2025].
- Han, J., Pei, J. and Tong, H. (2023) *Data Mining Concepts and Techniques (4th Edition)*. Fourth edition ed. [Online] : Elsevier. Available from: <https://app.knovel.com/hotlink/toc/id:kpDMCTE007/data-mining-concepts/data-mining-concepts?kpromoter=Summon>. [4/13/2025].
- Hastie, T., Tibshirani, R. and Friedman, J.H. (2011) *The elements of statistical learning*. 2. ed., corrected at 5. print. ed. [Online] New York [u.a.]: Springer. Available from: [http://bvbr.bib-bvb.de:8991/F?func=service&doc\\_library=BVB01&local\\_base=BVB01&doc\\_number=022644044&sequence=000004&line\\_number=0002&func\\_code=DB\\_RECORDS&service\\_type=MEDIA](http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&local_base=BVB01&doc_number=022644044&sequence=000004&line_number=0002&func_code=DB_RECORDS&service_type=MEDIA). [4/12/2025].
- Ibrahim, A.A., Ayilara-Adewale, O.A., Alabi, A.A. and Olusesi, D.A. (2025) Evaluation of Price Prediction of Houses in a Real Estate via Machine Learning. *Journal of Applied Sciences and Environmental Management*, [Online] 29 (1), pp. 43–48 Available from: <https://doaj.org/article/9fddfb5d90094a15a2db69017a8df818>. [4/13/2025].
- James, G., Witten, D., Hastie, T., Tibshirani, R. and Taylor, J.E. (2023) *An introduction to statistical learning*. 1st ed. [Online] Cham: Springer. Available from: <https://doi.org/10.1007/978-3-031-38747-0>. [4/14/2025].

Kim, J., Lee, Y., Lee, M. and Hong, S. (2022) A Comparative Study of Machine Learning and Spatial Interpolation Methods for Predicting House Prices. *Sustainability*, [Online] 14 (15), pp. 9056 Available from: <https://www.proquest.com/docview/2700786032>. [4/13/2025].

Krzywinski, M. and Altman, N. (2017) Classification and regression trees. *Nature methods*, [Online] 14 (8), pp. 757–758 Available from: <https://link.springer.com/article/10.1038/nmeth.4370>. [4/12/2025].

LeCun, Y., Bengio, Y. and Hinton, G. (2015) Deep learning. *Nature (London)*, [Online] 521 (7553), pp. 436–444 Available from: <https://link.springer.com/article/10.1038/nature14539>. [4/13/2025].

Li, M., Zhu, M., He, Y., Li, P., Zhang, P., He, Z., Shi, J., Chen, K., Weng, T., Zheng, Z. and Jiang, L. (2020) Analysis and Evaluation of Housing Price for Chengdu Urban. In: *ICCWAMTIP* . , [Online] Available from: <https://ieeexplore.ieee.org/document/9317395>. [4/14/2025].

Little, R.J.A. and Rubin, D.B. (2020) *Statistical analysis with missing data*. 3rd edition ed. [Online] Hoboken, NJ: Wiley. Available from: [http://bvbr.bib-bvb.de:8991/F?func=service&doc\\_library=BVB01&local\\_base=BVB01&doc\\_number=025207876&sequence=000001&line\\_number=0001&func\\_code=DB\\_RECORDS&service\\_type=MEDIA](http://bvbr.bib-bvb.de:8991/F?func=service&doc_library=BVB01&local_base=BVB01&doc_number=025207876&sequence=000001&line_number=0001&func_code=DB_RECORDS&service_type=MEDIA). [4/13/2025].

Mukesh, Gowda, H., Hr, A. and Yk, D. (2025) International Journal of Research Publication and Reviews. , [Online] , pp. Available from: <https://> . [4/13/2025].

Ranjith, S. and Ganesh, D. (2024) Housing Price Prediction using Machine Learning Technique. *International Journal of Advanced Research in Science, Communication and Technology*, [Online] , pp. 298–305 Available from: <https://doi.org/10.48175/ijarsct-15953>. [4/12/2025].

Ritu, R. (2015) *Machine learning techniques for house price prediction: A literature review*. ed. [Online] : .Available from: <https://> . [4/11/2025].

Supriya, M.S., Vinayak, G.S., Patgar, V.R. and Mahajan, V. (2023) House Price Prediction System using Machine Learning Algorithms and Visualization. In: *CONECCT* . , [Online] Available from: <https://ieeexplore.ieee.org/document/10234749>. [4/13/2025].

Tan, P., Steinbach, M., Karpatne, A. and Kumar, V. (2019) *Introduction to data mining*. 2nd ed ed. [Online] Harlow: Pearson. Available from: [https://ebookcentral.proquest.com/lib/\[SITE\\_ID\]/detail.action?docID=5720020](https://ebookcentral.proquest.com/lib/[SITE_ID]/detail.action?docID=5720020). [4/12/2025].